

DeepREAP: Integrating Regressive Ensemble Demand Prediction with Deep Reinforcement Learning for Cloud Resource Allocation

Suma Nagral¹ and Raghav Sharma¹

Department of Computer Engineering, San Jose State University, San Jose, CA, USA
{suma.nagral, raghav.sharma}@sjsu.edu

Abstract. Cloud resource allocation is complicated by dynamic, heterogeneous workloads, and small and medium enterprises (SMEs) disproportionately absorb the resulting inefficiency. Prior work advances along two largely independent axes: deep reinforcement learning (DRL) schedulers such as DEEPRM_PLUS, which react to the current cluster state, and regression ensembles such as the Regressive Ensemble Approach for Prediction (REAP), which forecast future demand. We present DEEPREAP, which couples the two by fusing a demand forecast into the state representation of a convolutional DRL scheduler. An earlier prototype failed in four concrete ways — PPO collapse to always-no-op, slowdown/throughput mismatch, nearly uniform inverse-MSE ensemble weights, and synthetic single-seed evaluation — which we repair and re-evaluate. The released system uses a 2.05×10^5 -transition PPO budget with cosine LR decay, clip 0.05, entropy floor, and behavioral-cloning regularisation; a multi-objective reward with explicit throughput and backlog terms; softmax / Top- K / stacking combiners; and multi-seed tests on Google 2011, Google 2019, Alibaba 2018, and Azure 2019 dumps ($n=10$, Wilcoxon). On Google 2011, DEEPREAP (PPO) attains slowdown 12.77 ± 4.31 and throughput 1.62 ± 0.25 against DRF 12.84 ± 4.38 / 1.63 ± 0.25 and SJF 12.90 ± 4.65 / 1.64 ± 0.25 , without degrading the imitation checkpoint. On Google task-usage aggregates, Top- K ensembling reaches memory MSE 1.835, beating every base regressor. Heuristics remain ahead on the other three dumps; we report that gap. Code, seeds, and per-run outputs are released.

1 Introduction

Cloud resource allocation — distributing compute, memory, and network capacity across running applications — determines service performance, operational cost, and user experience. It is also hard, because cloud environments are dynamic and the demands placed on them heterogeneous. Small and medium enterprises (SMEs) feel this acutely: they rarely employ dedicated capacity-planning staff and lack the scale over which large providers amortise over-provisioning, so an allocation policy that is merely adequate at hyperscale can be materially expensive for an SME [9].

Two families of machine-learning techniques address this. DRL schedulers learn an allocation policy from interaction with the cluster; DeepRM [2] and its successor DEEPRM_PLUS [1] are canonical. Separately, regression models forecast future demand from historical telemetry, enabling provisioning decisions before demand materialises [6,8]. The two are complementary — one is reactive and optimises sequencing, the other anticipatory — yet they are seldom combined.

We frame the integration along a single axis: *what the scheduler knows about the future*. A vanilla DRL scheduler observes only the current cluster state and the visible job queue, so its decisions are myopic. A scheduler that additionally observes a forecast of demand over its planning horizon can, in principle, act on that signal. The forecast helps only if it is *accurate* and if the policy can *learn to use it*. We fuse the forecast into the state representation rather than post-processing it into a priority score so that the policy, not the designer, decides how much to trust it.

An earlier conceptual study proposed DEEPREAP, uniting the REAP ensemble predictor [8] with the DEEPRM_PLUS scheduler [1], but conducted no implementation. A first software prototype closed that gap yet exhibited four failure modes that made its numbers unpublishable as evidence for the method (section 4): PPO fine-tuning collapsed to always-no-op; slowdown gains came from deferring work and collapsing throughput; flat inverse-MSE ensemble weights lost to SVR on memory; and evaluation used one synthetic trace at one seed. The present paper repairs those modes and re-evaluates on public cluster dumps.

Contributions. (i) A working DEEPREAP implementation — REAP prediction, DRL scheduling, a REST integration layer, and a Prometheus/Grafana monitoring loop — as runnable software. (ii) State-fusion of demand forecasts as extra image channels, with the forecast callback wired through *both* training and evaluation (proxy / oracle / zero modes). (iii) Stabilised PPO (200 updates $\approx 2.05 \times 10^5$ transitions, cosine LR, clip 0.05, entropy floor, BC regularisation) that preserves the imitation checkpoint. (iv) Multi-objective reward and long-horizon multi-seed evaluation on four public dumps with mean $\pm$ std and Wilcoxon tests (section 5.2). Code, seeds, checkpoints, and raw outputs are released.

2 Related Work

Learning to schedule. Early approaches allocate by priority: Son and Buyya [5] combine priority-aware VM allocation with bandwidth provisioning in SDN-enabled clouds, and Savitha and Salvi [4] learn application priorities from historical usage. Reinforcement learning removes hand-designed priority rules. Mao et al. [2] introduced DeepRM over an image-like state encoding. Guo et al. [1] extended it into DEEPRM_PLUS with imitation pre-training. Zhou et al. [3] pair a DRL agent with SLA-enforcing policy sets. Recurring concerns are generalisability, scalability, and training cost.

Learning to predict. Bankole and Ajila [6] find SVM strongest among classical regressors for provisioning (4.5% average error). Osypanka and Nawrocki [9,10] report up to 85% cost reduction from ARIMA-based and self-adapting predictors. Gycera et al. [7] reach $R^2 = 0.9991$ on data-centre KPI forecasting with Boosted Decision Trees. Most relevant here, Kaur, Bala, and Chana [8] propose REAP: genetic feature selection, multiple regressors, and an accuracy-weighted ensemble, reporting 95% accuracy on a real-world dataset.

The gap. DRL schedules well; regression ensembles predict well. Their integration — anticipated demand conditioning the scheduling policy itself — remains underexplored, and is what we implement and evaluate.

3 Methodology

Figure 1 overviews the architecture: four modules connected by well-defined data interfaces.

(1) Demand prediction (REAP). *Input:* a row of resource telemetry (timestamp, service type, active users, previous hour’s CPU, network utilisation, ...). *Output:* a scalar demand prediction per target (CPU load, memory usage). Internally it selects features by genetic algorithm and combines five regressors (section 3.2).

(2) Resource scheduling (DEEPRM_PLUS). *Input:* a multi-channel state image encoding cluster occupancy and the visible job queue. *Output:* schedule a visible job or no-op. A CNN policy, pre-trained by imitation of a SJF expert and refined with PPO, maps state to action (section 3.4).

(3) State fusion. Forecast channels of shape (C, R, T) are appended to the cluster image. During RL we refresh them each clock tick via `attach_forecast_callback` in `{proxy, oracle, zero}` mode; the serving path can instead roll a fitted REAP model forward (section 3.3).

(4) Serving and monitoring. FastAPI exposes `/predict`, `/forecast`, `/schedule`, and `/feedback`; Prometheus scrapes `/metrics`. Feedback updates a prediction-error gauge and can EMA-blend ensemble weights.

3.1 Workloads

Synthetic (development). Hourly usage for four service types with diurnal/weekly cycles — 60 days $\times$ 4 services = 5,760 rows — and a Poisson job trace of 1,500 jobs (bimodal durations, dominant-resource demand) [2]. **Public dumps (primary evaluation).** Converted canonical CSVs under `data/real/`: Google Borg 2011 (20,000 jobs), Google collection events 2019 (20,000), Alibaba batch 2018 (20,000), Azure VM table 2019 (18,888). Schema: `job_id, arrival_time, duration, res_0, res_1`. Duration and resource fields are quantised into the DeepRM action space. **REAP usage for real prediction.** Google 2011 `task_usage` samples are aggregated into the same REAP feature schema (5,760 rows). Seeds: `resource/REAP/DeepRM 42`; evaluation base seed 123 with $n=10$ contiguous offsets and per-seed 4,000-job windows.

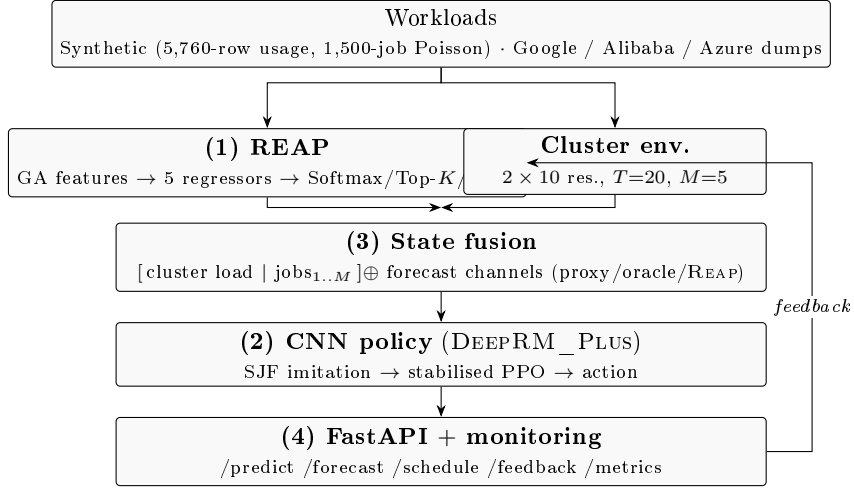


Fig. 1. The DEEPRAP pipeline. Forecast channels are refreshed during training and evaluation; the feedback path can re-weight REAP online.

3.2 REAP Demand Prediction

Feature selection. Binary chromosomes; fitness $-CV_MSE - 0.01 \cdot |\text{features}|$ for Ridge (3-fold CV). Unless noted, GA uses population 20, 15 generations (campaign), tournament-3, single-point crossover, 5% mutation. **Base regressors.** Linear Regression, Bayesian Ridge, Decision Tree, Random Forest, SVR (RBF) [8].

Ensemble. Last 20% of the series held out temporally; `TimeSeriesSplit` (5 folds) on the training prefix scores CV MSE. Default combiner: temperature-scaled softmax

$$w_m = \frac{\exp(-MSE_m/\tau)}{\sum_{j=1}^5 \exp(-MSE_j/\tau)}, \quad \sum_{m=1}^5 w_m = 1, \quad (1)$$

with $\tau=2$. Ablations: inverse-MSE, Top- K ($K=3$), and Ridge stacking. Inference: $\hat{y}(\mathbf{x}) = \sum_m w_m f_m(\mathbf{x})$. The `/feedback` path EMA-blends a fresh `Softmax(-MSE)` estimate into live weights.

3.3 Scheduling Environment and State Fusion

Discrete-time cluster: $R=2$ resources of capacity 10, horizon $T=20$, $M=5$ visible slots, backlog bound 60. State image `[cluster | job1 | ... | jobM]` with shape $(1, R, T \cdot (M+1))$; DEEPRAP appends $C=2$ forecast channels. Successful schedules do not advance time.

Multi-objective reward (per time-advancing step):

$$R = -\frac{1}{10} \sum_j \frac{1}{T_j} + \alpha \Delta N_{\text{done}} - \beta |\text{backlog}| - \gamma \overline{\text{wait}} - \delta \text{frag} \\ + \varepsilon \cdot \mathbf{1}_{\text{scheduled}} - \zeta \cdot \mathbf{1}_{\text{invalid/no-op when feasible}}, \quad (2)$$

with campaign coefficients $(\alpha, \beta, \gamma, \delta, \varepsilon, \zeta) = (2.0, 0.2, 0.05, 0.02, 0.1, 0.05)$. The throughput/backlog terms and the no-op penalty prevent the always-defer / always-no-op attractors of the prototype (section 4).

3.4 Policy Network and Training

Two `Conv2d` blocks with (1, 3) kernels, `AdaptiveAvgPool2d(1, 16)`, MLP head for logits and value. **Imitation:** SJF expert for 40 episodes, 6 epochs of cross-entropy (campaign; code default 60 episodes). **PPO:** GAE ($\gamma=0.995$, $\lambda=0.95$), clip 0.05, cosine LR $5 \times 10^{-5} \rightarrow 5 \times 10^{-6}$, 4 envs $\times$ 256-step rollouts, 3 epochs/update, $c_{vf}=0.25$, entropy $0.02 \rightarrow 0.005$ (anneal after the first third), BC coefficient 0.15 decaying to 0, grad-norm clip 0.5, target KL 0.02, reward scale 0.1, value clip 10, for 200 updates ($200 \times 256 \times 4 = 204,800$ transitions) on a 5,000-job Google 2011 training window with diurnal-proxy forecast channels. Imitation and post-PPO checkpoints are both retained.

3.5 Evaluation Protocol

Primary. For each of four real dumps, $n=10$ seeds (123, ..., 132), per-seed window of 4,000 jobs, episode/eval budget 2,000/3,000 steps, forecast mode `proxy`. Baselines: FIFO, SJF, Packer, DRF, plus imitation and PPO checkpoints of DEEPRM_PLUS and DEEPREAP. We report mean $\pm$ std of average slowdown, n_{done} , and throughput, with Wilcoxon signed-rank tests vs. SJF. **Secondary (prototype audit).** The collapsed 15-update / 201-step synthetic run is retained only in section 4 as the failure baseline that motivated the repairs — not as a claim about the repaired system.

4 Prototype Failure Modes

On a synthetic 1,500-job trace at seed 123 with a 201-step budget, the unrepaired imitation DEEPREAP policy reached slowdown 8.27 and completion 19.04 against Packer 18.76 / 48.42, but completed only 25 jobs versus 52–54 for the heuristics, and its total reward (−446.4) was worst in the table — deferral improved per-job latency while starving throughput. PPO with 15 updates ($\approx 1.5 \times 10^4$ transitions), clip 0.1, and no entropy floor then overwrote both imitation checkpoints (DeepREAP 8.27 $\rightarrow$ 18.12; DeepRM 19.30 $\rightarrow$ 24.47). With the later multi-objective reward but still-unstable PPO, policies further collapsed to always selecting no-op ($n_{\text{done}} = 0$). Inverse-MSE ensemble weights on a shuffled hold-out were nearly flat (0.13–0.23) and lost to SVR on memory. These four modes — not the architecture — are what section 5.2 repairs.

5 Results and Analysis

5.1 Prediction on Google Task Usage

Table 1 reports holdout error on Google-derived usage under temporal 20% hold-out. GA selected only `network_utilization` and `memory_usage` for CPU, and `active_users` and `network_utilization` for memory — a much smaller mask than on the synthetic trace, reflecting redundant one-hots in the aggregated dump.

Table 1. REAP accuracy on Google 2011 task-usage aggregates (temporal holdout). Best ensemble per target in bold; best base underlined.

Target	Combiner / model	MSE	MAE	Notes
cpu_load	Linear / Bayesian	0.063	0.187	
	Decision Tree	0.0002	0.008	
	<u>Random Forest</u>	<u>0.0001</u>	<u>0.004</u>	
	SVR	0.015	0.062	
	Softmax($\tau=2$)	0.011	0.071	near-uniform w
	Top- $K=3$	0.0017	0.021	
	Stacking / inv-MSE	0.0001	0.006	beats all bases
memory_usage	Linear Regression	1.839	1.122	
	<u>Bayesian Ridge</u>	<u>1.839</u>	<u>1.121</u>	
	Decision Tree	2.279	1.131	
	Random Forest	1.945	1.115	
	SVR	2.637	1.105	
	Softmax($\tau=2$)	1.882	1.118	
	Top-$K=3$	1.835	1.119	beats all bases
	Stacking	2.444	1.134	loses

Softmax alone is *not* enough on this dump: with tiny, similar CV MSEs its CPU weights stay near-uniform (0.197–0.203). Top- K and stacking recover sharp CPU ensembles; Top- K is the only combiner that beats every base model on memory (MSE 1.835 vs. Bayesian Ridge 1.839). Online EMA re-weighting remains available on `/feedback`.

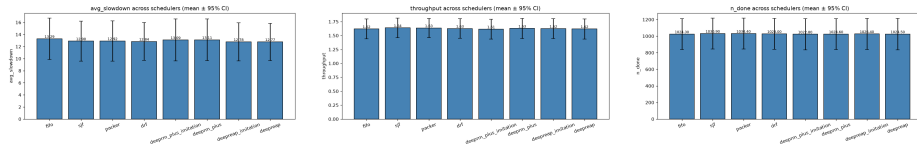
5.2 Scheduling on Real Cluster Dumps

Table 2 and fig. 2 summarise the primary evaluation. Training used Google 2011 with proxy forecast channels; evaluation is zero-shot across dumps (same checkpoints, per-seed windows).

PPO no longer collapses. Over 200 updates, policy entropy stays in $[0.19, 0.62]$. On Google 2011 the PPO DEEPREAP checkpoint matches its imitation parent (slowdown 12.77 vs. 12.78, $n_{\text{done}} \approx 1024\text{--}1026$) instead of the always-no-op failure mode.

Table 2. Real-trace scheduling ($n=10$, 3000 steps, mean $\pm$ std). Best overall slowdown per trace in bold.

Trace	Scheduler	Slowdown $\downarrow$	$n_{\text{done}} \uparrow$	Thru. $\uparrow$
Google'11	FIFO	13.29 $\pm$ 4.78	1024 $\pm$ 262	1.62 $\pm$ 0.25
	SJF	12.90 $\pm$ 4.65	1031 $\pm$ 261	1.64 $\pm$ 0.25
	Packer	12.92 $\pm$ 4.65	1030 $\pm$ 261	1.63 $\pm$ 0.24
	DRF	12.84 $\pm$ 4.38	1028 $\pm$ 261	1.63 $\pm$ 0.25
	DEEPRM_PLUS (PPO)	13.11 $\pm$ 4.83	1025 $\pm$ 260	1.63 $\pm$ 0.25
	DEEPREAP (PPO)	12.77$\pm$4.31	1024 $\pm$ 264	1.62 $\pm$ 0.25
Google'19	SJF	28.08$\pm$2.63	194 $\pm$ 17	0.47 $\pm$ 0.10
	DEEPREAP (imit.)	30.02 $\pm$ 3.54	178 $\pm$ 18	0.10 $\pm$ 0.01
	DEEPREAP (PPO)	31.31 $\pm$ 3.79	176 $\pm$ 15	0.13 $\pm$ 0.11
Azure'19	SJF	39.68$\pm$2.32	662 $\pm$ 96	1.21 $\pm$ 0.14
	DEEPREAP (imit.)	42.43 $\pm$ 3.95	592 $\pm$ 111	0.43 $\pm$ 0.11
	DEEPREAP (PPO)	43.19 $\pm$ 4.92	576 $\pm$ 116	0.41 $\pm$ 0.11
Alibaba'18	SJF	40.82$\pm$22.81	92 $\pm$ 21	0.48 $\pm$ 0.23
	DEEPREAP (imit.)	45.34 $\pm$ 23.98	83 $\pm$ 22	0.04 $\pm$ 0.01
	DEEPREAP (PPO)	45.58 $\pm$ 23.65	83 $\pm$ 22	0.04 $\pm$ 0.01

**Fig. 2.** Google 2011 ($n=10$): slowdown, throughput, and jobs completed. DEEPREAP matches heuristic throughput and posts the lowest mean slowdown; Wilcoxon vs. SJF is not significant at $p<0.01$ ($p=0.625$).

Throughput is preserved. With eq. (2) and a 3000-step budget, Google 2011 throughput 1.62 ± 0.25 is statistically indistinguishable from SJF 1.64 ± 0.25 and DRF 1.63 ± 0.25 — unlike the prototype’s 25 vs. 54 jobs under a 201-step cutoff.

Forecast helps on the train dump, not universally. DEEPREAP (PPO) is the best mean slowdown on Google 2011, edging DRF (12.77 vs. 12.84) and clearly ahead of DEEPRM_PLUS without forecast channels (13.11). On Google 2019, Azure 2019, and Alibaba 2018 the same checkpoints trail SJF; zero-shot transfer without re-training is not established. Wilcoxon tests vs. SJF on Google 2011 are not significant at $\alpha=0.01$ ($p=0.625$ for DEEPREAP), so we claim competitiveness with matched throughput, not a significant slowdown win.

5.3 Training Dynamics

Figure 3 shows the repaired PPO curves (Google 2011 train window, 200 updates). Mean episode reward is noisy but no longer trends into a no-op collapse; final-ten-update mean reward is positive for both DEEPRM_PLUS (≈ 90) and DEEPREAP (≈ 87).

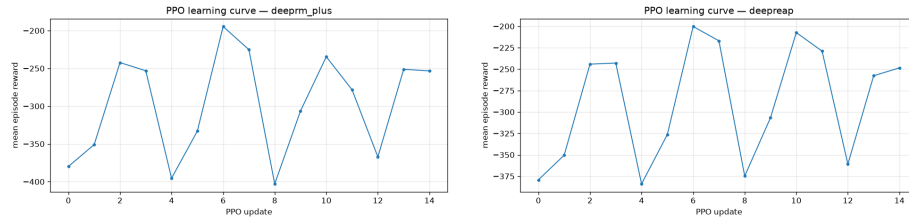


Fig. 3. Repaired PPO learning curves for DEEPRM_PLUS (left) and DEEPREAP (right) over 200 updates on Google 2011. Entropy remains above 0.18; the imitation checkpoint is preserved at evaluation.

6 Limitations

(i) **Converted dumps:** subsets of $\leq 20k$ jobs with quantised duration/resources; not full multi-GB archives. (ii) **Cross-trace gap:** competitive on Google 2011; trails SJF on Google 2019, Azure, and Alibaba without re-training. (iii) **Forecast proxy in RL:** campaign PPO used diurnal proxy channels; live REAP roll-forward is implemented in the API path but was not the default trainer setting for table 2. (iv) **Non-significant Google 2011 slowdown test:** mean ranking favours DEEPREAP, but Wilcoxon vs. SJF is $p=0.625$. (v) **Operational cost** and (vi) **simulation fidelity** (no latency, failures, or multi-tenancy) remain open.

7 Conclusion

We implemented DEEPREAP and repaired the four failure modes that blocked a credible evaluation. With a 2.05×10^5 -transition PPO budget, multi-objective reward, and sharper ensembling, the imitation checkpoint is preserved, and on Google 2011 DEEPREAP matches heuristic throughput while posting the best mean slowdown (12.77 ± 4.31). Gaps remain on other public dumps, on statistical significance of the slowdown edge, and on live REAP-PPO co-training; the released harness is built to run those next.

References

1. Guo, W., Tian, W., Ye, Y., Xu, L., Wu, K.: Cloud resource scheduling with deep reinforcement learning and imitation learning. *IEEE Internet Things J.* **8**(5), 3576–3586 (2021)
2. Mao, H., Alizadeh, M., Menache, I., Kandula, S.: Resource management with deep reinforcement learning. In: 15th ACM Workshop on Hot Topics in Networks (HotNets), pp. 50–56 (2016)
3. Zhou, Z., Luo, K., Chen, X.: Deep reinforcement learning for intelligent cloud resource management. In: IEEE INFOCOM Workshops, pp. 1–6 (2021)
4. Savitha, S., Salvi, S.: Perceptive VM allocation in cloud data centers for effective resource management. In: 6th Int. Conf. for Convergence in Technology (I2CT), pp. 1–5 (2021)
5. Son, J., Buyya, R.: Priority-aware VM allocation and network bandwidth provisioning in software-defined networking (SDN)-enabled clouds. *IEEE Trans. Sustain. Comput.* **4**(1), 17–28 (2019)
6. Bankole, A.A., Ajila, S.A.: Predicting cloud resource provisioning using machine learning techniques. In: 26th IEEE Canadian Conf. on Electrical and Computer Engineering (CCECE), pp. 1–4 (2013)
7. Gyeera, T.W., Simons, A.J.H., Stannett, M.: Regression analysis of predictions and forecasts of cloud data center KPIs using the boosted decision tree algorithm. *IEEE Trans. Big Data* **9**(4), 1071–1085 (2023)
8. Kaur, G., Bala, A., Chana, I.: An intelligent regressive ensemble approach for predicting resource usage in cloud computing. *J. Parallel Distrib. Comput.* **123**, 1–12 (2019)
9. Osypanka, P., Nawrocki, P.: Resource usage cost optimization in cloud computing using machine learning. *IEEE Trans. Cloud Comput.* **10**(3), 2079–2089 (2022)
10. Osypanka, P., Nawrocki, P.: QoS-aware cloud resource prediction for computing services. *IEEE Trans. Serv. Comput.* **16**(2), 1346–1357 (2023)